

Dossier Space Pilot: Complete Infrastructure Architecture

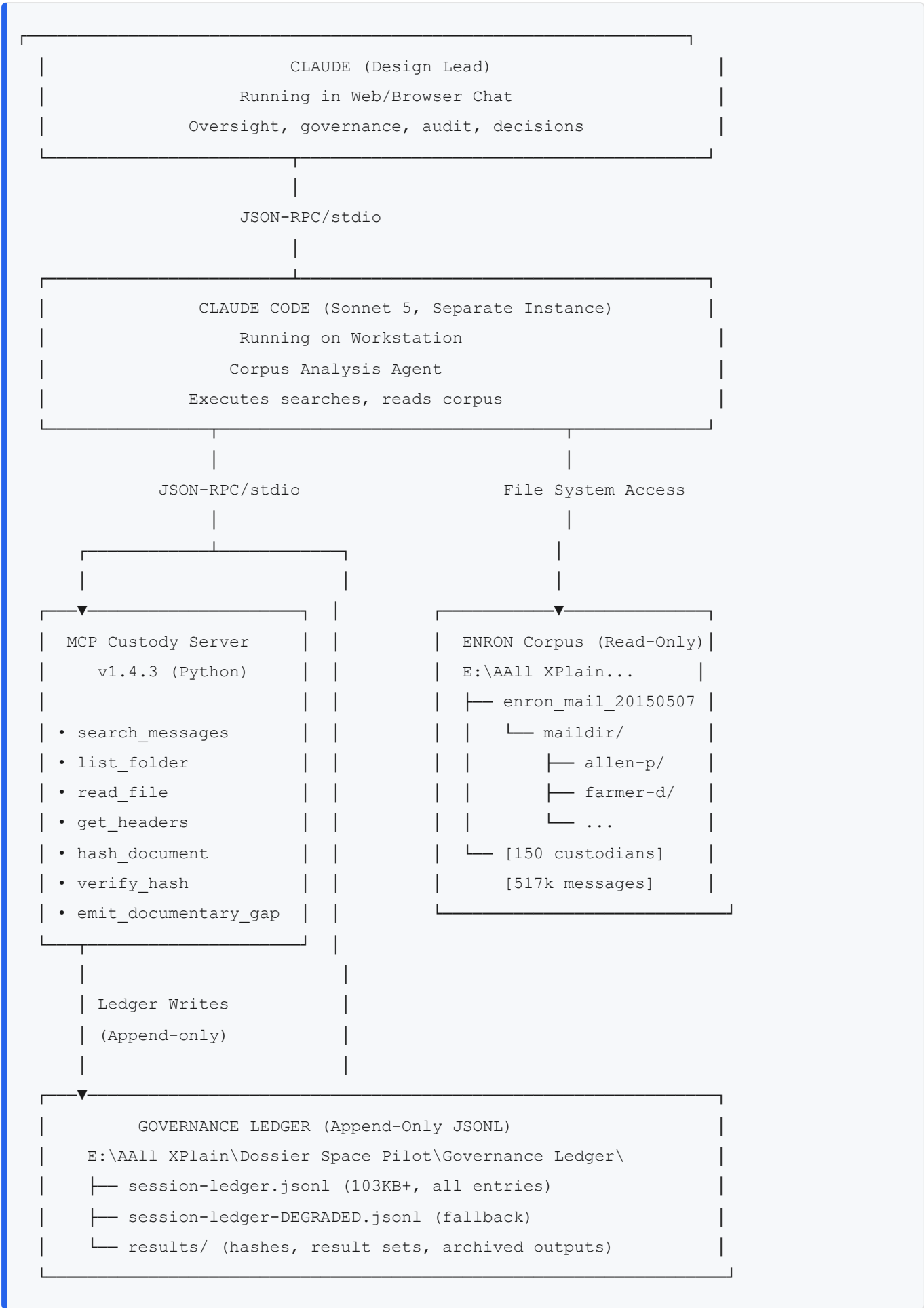
Date: 2026-08-17

Version: 1.0 (Post-Repair)

Scope: File structure, data flows, governance, component interactions, design rationale

Audience: Ken Tombs (Operator), Claude (Design Lead), Leonard (External Assurance)

ARCHITECTURE OVERVIEW



DIRECTORY STRUCTURE (Complete)

Root: `E:\AAll XPlain\Dossier Space Pilot`

Dossier Space Pilot/	
	└─ ENRON Data Set Test 1/ [Corpus + Governance]
	└─ enron_mail_20150507/ [CORPUS – Read-Only]
	└─ maildir/ [517,401 emails, 150 custodians]
	└─ allen-p/ [Phillip Allen]
	└─ farmer-d/ [David Farmer]
	└─ kaminski-v/ [Vince Kaminski]
	└─ whalley-g/ [Greg Whalley]
	└─ ... [146 more] [Total 150 custodian folders]
	└─ Governance Ledger/ [LEDGER – Append-Only Governance]
	└─ session-ledger.jsonl [103KB, all entries A01-A09+]
	└─ session-ledger-DEGRADED.jsonl [Fallback if primary fails]
	└─ results/ [Archived result sets, hashes]
	└─ session_11_audit_log.txt [Session 11 notes]
	└─ session_12_audit_log.txt [Session 12 notes]
	└─ session_12_design_lead_log.txt [Claude notes]
	└─ findings/ [Investigation Outputs]
	└─ F-001-*.md [ROMER-format findings]
	└─ ... [findings from Sessions 5-12]
	└─ transcripts/ [Session Conversations]
	└─ [Various transcript exhibits]
	└─ git-repo-exhibit-2026-07-15/ [Archived Git Deliverables]
	└─ recovered-artefacts-exhibit/ [Historical Artifacts]
	└─ run12B-outputs-exhibit/ [Session Outputs]
	└─ session5-* exhibits/ [Historical Sessions]
	└─ subject-memory-exhibit/ [Claude Code Memories]
	└─ ADR-001 through ADR-004 [Architecture Decision Records]
	└─ ADR-001-architecture.docx [Custody model]
	└─ ADR-002-artefact-architecture.docx
	└─ ADR-003 (implied) [Governance model]
	└─ ADR-004 (implied) [Recorded-direct posture]
	└─ Custody Server/ [MCP SERVER CODE]
	└─ dossier_custody_server_v1.2.py [Archive: v1.2]

```

|   ├── dossier_custody_server_v1.3.py    [Archive: v1.3 (no JSON-RPC)]
|   ├── dossier_custody_server_v1_4.py    [Archive: v1.4 (FastMCP)]
|   ├── dossier_custody_server_v1_4_1.py  [Archive: v1.4.1]
|   ├── dossier_custody_server_v1_4_2.py  [Archive: v1.4.2]
|   ├── dossier_custody_server_v1_4_3.py  [✅ PRODUCTION – Currently Running]
|   ├── dossier_custody_server_v1_4_4.py  [Archive: v1.4.4 (raw JSON-RPC, untested)]
|   ├── dossier_custody_server.py          [Link/shorthand to active version]
|   ├── smoke_test_v1_4.py                [Test harness]
|   └── [Backup files from edits]
|
└── Session GIT Deliverables/              [Outputs to GitHub/External]
    ├── [Session-specific Git repos]
    └── [For publication with SCL]

```

Local Config: `C:\Users\ktomb.claude`

```

.claude/
└── claude.json                        [MCP Server Registration]
    ├── dossier-custody server config
    │   ├── command: "python"
    │   ├── args: ["E:...\dossier_custody_server_v1_4_3.py"]
    │   └── type: "stdio"
    └── [Google service configs: Gmail, Calendar, Drive]

```

KEY FILES: PURPOSE & RATIONALE

1. CORPUS

File: `E:\AAll XPlain\Dossier Space Pilot\ENRON Data Set Test 1\enronmail20150507\maildir`

What It Is:

- CMU Enron Email Archive (extracted)
- 517,401 email messages
- 150 custodians (person/role folders)
- Frozen point-in-time, immutable

Why We Have It:

- Test corpus for the pilot (standardized, well-studied, publicly available)
- Sovereignty design requirement: data stays on workstation, never leaves
- Allows testing of evidence analysis without real data risk

How It's Used:

- Claude Code reads .eml files via MCP Custody Server tools
- search_messages queries custodians by keyword
- read_file retrieves full message content
- get_headers extracts metadata
- hash_document computes integrity anchor

Governance:

- Read-only (never modified by pilot code)
 - All accesses logged to governance ledger (action_id, hash, result)
 - Hashable (SHA256 per file) for tampering detection
-

2. GOVERNANCE LEDGER

Files:

- `session-ledger.jsonl` — Primary ledger (103KB, all entries)
- `session-ledger-DEGRADED.jsonl` — Fallback (if primary write fails)

What It Is:

- Append-only JSON Lines file
- One entry per line, never edited (only appended)
- Full audit trail of all operations
- Fail-closed: if write fails on primary, attempts fallback; if both fail, process halts

Why We Have It:

- Core governance requirement for the pilot
- "Watch is never the watched" — all agent actions independently recorded
- Enables audit and chain-of-custody for published findings
- Required for Society for Computers and Law paper

Structure (Each Entry):

```

{
  "ledger_entry_id": "<UUID>",          # Unique ID for this entry
  "session_id": "DS-202608xx-S12",     # Which session/operator
  "task_id": "DS-202608xx-S12-T01",    # Which task within session
  "action_id": "DS-202608xx-S12-A09",  # Sequential action counter
  "entry_type": "tool_call",           # Type: session_start, tool_call, documenta
  ry_gap, etc.
  "timestamp": "2026-08-17T15:59:00Z", # When it happened (UTC)
  "operator": "Ken Tombs",             # Who initiated
  "description": "Search messages...",  # Human-readable summary
  "material_decision": false,          # Did this change the investigation?
  "severity": "informational",         # critical/major/minor/informational
  "payload": {                        # Tool-specific data
    "tool_name": "search_messages",
    "parameters": {...},
    "result_set_count": 42,
    "tool_output": {...}
  },
  "status": "active",                 # active/superseded
  "superseded_by": null,              # Link to newer entry if updated
  "reconciliation_notes": ""          # Audit notes if corrected
}

```

How It's Used:

- Every tool call auto-logs here (*searchmessages*, *readfile*, *emitdocumentarygap*, etc.)
- Claude Code has NO permission to edit it (read-only from outside)
- Only the Custody Server appends to it
- Provides immutable proof of what was examined

Controlled Value Enums:

- *entrytype*: *sessionstart*, *decision*, *toolcall*, *mcpcall*, *resultset*, *documentarygap*, etc.
- *severity*: *critical*, *major*, *minor*, *informational*
- *status*: *active*, *superseded*

3. MCP CUSTODY SERVER v1.4.3

File: `E:\AA11 XPlain\Dossier Space Pilot\Custody Server\dossier_custody_server_v1_4_3.py`

What It Is:

- Python server exposing 7 tools via JSON-RPC over stdio
- Runs as child process of Claude Code
- Bridges Claude Code ↔ Corpus + Ledger

Why We Have It:

- Sovereignty requirement: no cloud calls, all work local
- Custody requirement: server independently logs all actions
- Mediates access: Claude Code cannot directly touch corpus or ledger

The 7 Tools (What Each Does):

Tool	Purpose	Logs	Reads From
search_messages	Find emails by keyword/date/custodian	Action, count	Corpus maildir
list_folder	List all .eml files in custodian folder	✓ Action, count	Corpus maildir
read_file	Get full message content (headers + body)	✓ Action, bytes	Corpus maildir
get_headers	Extract headers only (From, To, Date, etc.)	✓ Action, header count	Corpus maildir
hash_document	Compute SHA256 of a file	✓ Action, hash	Corpus maildir
verify_hash	Compare file's current hash to expected	✓ Action, match result	Corpus maildir
emitdocumentarygap	Log a missing doc/communication to ledger	✓ Action, gap entry	Governance Ledger

Architecture (Inside the Server):

dossier_custody_server_v1_4_3.py

|
|— Imports

| |— os, sys, json (file I/O)
| |— logging (console output)
| |— asyncio (async message loop)

|
|— Configuration (Top of File)

| |— SESSION_ID = "DS-202608xx-S12"
| |— ENRON_CORPUS_ROOT = "E:..."
| |— LEDGER_FILE = "E:...\\session-ledger.jsonl"
| |— ENTRY_TYPES = {set of valid entry types}
| |— SEVERITY_LEVELS = {critical, major, minor, informational}

|
|— LedgerWriter Class

| |— __init__() - Initialize ledger paths, recover action counter
| |— _recover_action_counter() - Read existing ledger, find highest action number
| |— next_action_id() - Generate next sequential action ID
| |— _validate_entry() - Check entry has valid types/severity/status
| |— _write_ledger() - Append JSON line to ledger file (primary + fallback)
| |— emit_entry() - Full ledger entry creation with fail-closed semantics

|
|— CustodyServer Class

| |— __init__() - Set corpus root, ledger reference, task ID
| |— _compute_document_hash() - SHA256 file hashing
| |— search_messages() - Keyword search across custodian
| |— list_folder() - List files in custodian
| |— read_file() - Read .eml file content
| |— get_headers() - Extract email headers
| |— hash_document() - Compute file hash
| |— verify_hash() - Compare hash to expected
| |— emit_documentary_gap() - Record gap to ledger

|
|— JSONRPCServer Class (Connection Handler)

| |— handle_message() - Parse incoming JSON-RPC request
| |— handle_initialize() - Respond to MCP initialize
| |— handle_tools_list() - Return array of 7 tools (schemas)
| |— handle_tool_call() - Route tool calls to CustodyServer methods
| |— error_response() - JSON-RPC error format

|
|— TOOLS_REGISTRY (List of 7 Tool Schemas)

| |— Each tool has: name, description, inputSchema (JSON)

|
|— message_loop() - Async stdio reader

```

|   |─ Read line from stdin (Claude Code sends JSON-RPC request)
|   |─ Parse JSON, validate
|   |─ Call handle_message()
|   |─ Write response to stdout (Claude Code receives result)
|   |─ Repeat
|
|─ main() - Entry Point
    |─ Create LedgerWriter (initialize ledger)
    |─ Create CustodyServer (initialize corpus access)
    |─ Log session_start to ledger
    |─ Create JSONRPCServer
    |─ Start message_loop() (wait for Claude Code requests)

```

Communication Pattern (JSON-RPC 2.0 over stdio):

Claude Code	Custody Server
─ INITIALIZE (JSON-RPC) —————>	
	─ handshake
<─ INITIALIZE response —————	
─ tools/list (JSON-RPC) —————>	
<─ [7 tools with schemas] —————	
─ tools/call: search_messages ———>	
	─ execute search
	─ log to ledger (A05)
<─ {"results": [...]} —————	
─ tools/call: emit_documentary_gap>	
	─ validate gap data
	─ emit to ledger (A09)
<─ {"gap_id": "...", "status": ...}	
─ [repeat for each operation]———	

Why This Architecture:

1. Separation of Concerns:

- Claude Code = Analysis agent (what to do)
- Custody Server = Execution + Logging (how to do it safely)
- They communicate via narrow JSON-RPC interface

2. Fail-Closed Semantics:

- If server crashes, Claude Code knows (stdio closes)

- If ledger write fails, process halts (don't hide failures)
- No silent errors, no skipped logging

3. Auditability:

- Every action logged independently by server
- Claude Code cannot modify or bypass logging
- Ledger is the canonical truth, not Claude Code's memory

4. Portability:

- Paths are absolute (E:\ drive) but could be made portable
- No hardcoded user info
- No external dependencies (no APIs, no cloud)

4. CLAUDE.JSON (MCP Registration Config)

File: `C:\Users\ktomb.claude\claude.json`

What It Is:

- Claude Code's MCP server configuration
- Lists all MCP servers Claude Code should launch

Why We Have It:

- Tells Claude Code: "When you start, launch this Python script and connect via stdio"
- Also configures Google service connectors (Gmail, Calendar, Drive)

Structure:

```
{
  "mcpServers": {
    "dossier-custody": {
      "command": "python",
      "args": ["E:\\AA11 XPlain\\Dossier Space Pilot\\Custody Server\\dossier_custody_server_v1_4_3.py"],
      "type": "stdio"
    },
    "claude_ai_Gmail": {...},
    "claude_ai_Google_Calendar": {...},
    "claude_ai_Google_Drive": {...}
  }
}
```

How Registration Works:

1. **Edit `claude.json`** — Add server config
2. **Run `claude mcp add`** — Register with Claude Code's MCP manager ⚠️ (CRITICAL STEP)

3. **Restart Claude Code** — Load the registered server
4. **Verify** `claude mcp list` — Confirm server appears
5. **Tools appear in deferred list** — Claude Code can now call them

Why Both File + Command Needed:

- File alone isn't enough; Claude Code needs explicit registration
- The `mcp add` command adds it to Claude Code's active registry
- Without registration, Claude Code ignores the config file

5. VERSION HISTORY (Archive Files)

Files: `dossier_custody_server_v1*.py`

Why We Keep Them:

- Audit trail of what changed and when
- Can revert if new version fails
- Shows evolution of the design (v1.3 → v1.4 was major rebuild)
- Helps debug regressions ("it worked in v1.2, let's see what changed")

Key Milestones:

Version	Date	Change	Status
v1.2	06/08	Initial custody server	Archive
v1.3	06/08	First JSON-RPC attempt (blocker: no real protocol)	Archive
v1.4	11/08	Clean rebuild with FastMCP 1.29.0	Archive
v1.4.1	13/08	Added <code>documentarygap entrytype</code>	Archive
v1.4.2	13/08	Added <code>emitdocumentarygap()</code> tool	Archive
v1.4.3	13/08	Fixed syntax error (line 736), registered with <code>mcp add</code>	✅ PRODUCTION
v1.4.4	13/08	Raw JSON-RPC, no FastMCP (untested, doesn't connect)	Archive

DATA FLOWS

Flow 1: Corpus Search → Ledger Logging

Claude Code

```
|
| └─ User: "search kaminski-v for 'Raptor'"
|
| └─ Calls: search_messages(custodian="kaminski-v", keyword="Raptor")
|         (JSON-RPC to Custody Server)
```

Custody Server

```
|
| └─ search_messages() executes:
|   └─ Validates: kaminski-v folder exists
|   └─ Searches: E:...\\enron_mail_20150507\\maildir\\kaminski-v*.eml
|   └─ Finds: 23 files matching "Raptor"
|   └─ Returns: list of file paths
|
| └─ Auto-logs to ledger:
|   └─ action_id = "DS-202608xx-S12-A05" (next sequential)
|   └─ entry_type = "tool_call"
|   └─ tool_name = "search_messages"
|   └─ parameters = {custodian: "kaminski-v", keyword: "Raptor"}
|   └─ result_set_count = 23
|   └─ Appends to session-ledger.jsonl
|
| └─ Returns to Claude Code:
|   └─ {"custodian": "kaminski-v", "count": 23, "results": [...]}
```

Claude Code

```
|
| └─ Receives results
| └─ Analyzes 23 files
| └─ Can call: read_file(results[0]), get_headers(results[1]), etc.
|
| └─ (Each call also logs to ledger)
```

Ledger Entry Created:

```
{  
  "ledger_entry_id": "<UUID>",  
  "action_id": "DS-202608xx-S12-A05",  
  "entry_type": "tool_call",  
  "tool_name": "search_messages",  
  "parameters": {"custodian": "kaminski-v", "keyword": "Raptor"},  
  "result_set_count": 23,  
  "timestamp": "2026-08-17T15:03:40Z"  
}
```

Flow 2: Documentary Gap Logging

Claude Code

```
|
| └─ During analysis, discovers a gap:
|   "August 22, 2001 meeting mentioned in Oct 4 memo, but not found"
|
| └─ Calls: emit_documentary_gap(
|     gap_identified="Aug 22 meeting content",
|     reason_identified="Referenced in Oct 4 memo, no independent record",
|     significance="high",
|     status="open"
|   )
|
```

Custody Server

```
|
| └─ emit_documentary_gap() executes:
|   └─ Validates: significance in [high, medium, low], status in [open, closed]
|   └─ Generates: gap_id (UUID)
|   └─ Returns immediately to Claude Code
|
| └─ Auto-logs to ledger:
|   └─ action_id = "DS-202608xx-S12-A09"
|   └─ entry_type = "documentary_gap" (special type)
|   └─ description = "Documentary gap: Aug 22 meeting content"
|   └─ payload.gap_identified = "Aug 22 meeting content"
|   └─ payload.reason_identified = "Referenced in Oct 4 memo, no independent reco
rd"
|   └─ payload.significance = "high"
|   └─ payload.status = "open"
|   └─ Appends to session-ledger.jsonl
|
```

Claude Code

```
|
| └─ Continues analysis; gap is now officially recorded
|     (Can reference action_id: DS-202608xx-S12-A09 in findings)
```

Ledger Entry Created:

```
{
  "ledger_entry_id": "<UUID>",
  "action_id": "DS-202608xx-S12-A09",
  "entry_type": "documentary_gap",
  "description": "Documentary gap: Aug 22 meeting content",
  "payload": {
    "gap_identified": "Aug 22 meeting content",
    "reason_identified": "Referenced in Oct 4 memo, no independent record",
    "significance": "high",
    "status": "open"
  },
  "timestamp": "2026-08-17T15:03:40Z"
}
```

Flow 3: File Hashing → Integrity Verification

Claude Code

```
|
| └─ Wants to verify file hasn't changed:
|   "I read kaminski-v/discussion_threads/7832.eml in Session 12,
|     has it been modified since then?"
|
| └─ Calls: verify_hash(
|     filepath="kaminski-v/discussion_threads/7832.eml",
|     expected_hash="abc123def456..." (from Session 12)
|   )
|
```

Custody Server

```
|
| └─ verify_hash() executes:
|   └─ Reads: E:...\maildir\kaminski-v\discussion_threads\7832.eml
|   └─ Computes: SHA256 hash
|   └─ Compares: computed vs expected
|   └─ Returns: {match: true/false, computed: "...", expected: "..."}
|
| └─ Auto-logs to ledger:
|   └─ entry_type = "tool_call"
|   └─ tool_name = "verify_hash"
|   └─ payload.match = true/false
|     └─ payload.computed, payload.expected
|
```

Claude Code

```
|
| └─ If match=false, file has changed (tamper detection)
|     If match=true, integrity verified
```

Use Case: Chain of custody — prove the file you're analyzing now is the same one from earlier sessions.

GOVERNANCE MODEL

Entry Types (What Gets Logged)

session_start	- Session initialized, server launched
decision	- A choice point (e.g., which custodian to analyze next)
tool_call	- A tool was executed (search, read, hash, etc.)
mcp_call	- An MCP protocol message (initialize, list, etc.)
result_set	- Results persisted to disk
candidate_set	- Candidate entities identified (emails, threads, etc.)
documentary_gap	- A missing doc/communication flagged
operator_intervention	- Human (Ken) made a manual decision
finding	- A substantive finding recorded (usually in findings/ file)
assurance	- External assurance/QA action
exception	- Error or failure condition
correction	- A ledger entry was corrected/superseded
session_close	- Session ended

Action ID Generation

DS-202608xx-S12-A09

└ DS	= Dossier Space (project)
└ 202608xx	= Session date (yyyymm, xx = placeholder for dd/version)
└ S12	= Session 12 (which investigation session)
└ A09	= Action 9 (sequential counter within session)

Counter recovers from ledger on startup:

- Read all existing entries in this session_id
- Find highest A-number
- Resume from there
- Prevents collisions if server restarts

Fail-Closed Ledger Write

```
CustodyServer.emit_entry() {  
    1. Validate: entry_type, severity, status in allowed enums  
    2. Try: Write to primary ledger file  
        If success → return immediately  
    3. If primary fails:  
        Log warning  
        Try: Write to fallback ledger file  
        If success → return  
    4. If both fail:  
        Log CRITICAL error  
        Raise RuntimeError (process halts)  
        Do NOT continue silently  
}
```

Why Fail-Closed?

- Never hide failures
 - If you can't record an action, don't proceed (conservatism)
 - Prevents situations where "we did something but forgot to log it"
-

WHY THIS ARCHITECTURE

1. Sovereignty (Data Stays Local)

Requirement: No cloud calls, no external API access, all data on workstation.

Implementation:

- Corpus: local filesystem (E:\ drive)
- Ledger: local filesystem (E:\ drive)
- MCP Server: Python subprocess on workstation
- All connections: stdio (not network)

Benefit: Complete control, no data leaves the building, complies with evidence security.

2. Chain of Custody (Every Action Logged)

Requirement: Prove what was examined, when, and by whom.

Implementation:

- LedgerWriter appends every tool call
- Each entry has: timestamp, operator, action_id, parameters, results
- Entries never edited (append-only)
- Ledger is immutable proof

Benefit: If questioned later ("Did you really examine that file?"), the ledger proves it.

3. Separation of Concerns (Agent ≠ Logger)

Requirement: Claude Code (agent) can't modify the ledger or bypass logging.

Implementation:

- Claude Code: Read corpus via MCP, analyze, propose findings
- Custody Server: Execute operations, independently log them
- Ledger: Custody Server only entity that writes

Benefit: Even if Claude Code is compromised or buggy, the ledger is honest.

4. Testability (Layers Isolated)

Requirement: Debug easily, test each layer independently.

Implementation:

- Test ledger: run `dossier_custody_server.py` standalone, check session-ledger.jsonl
- Test tools: call each tool, verify results
- Test connection: `claude mcp list` shows registered servers
- Test end-to-end: Claude Code calls tool, ledger updates

Benefit: Rapid diagnosis (as we did today with smoke tests).

5. Extensibility (Add Tools Later)

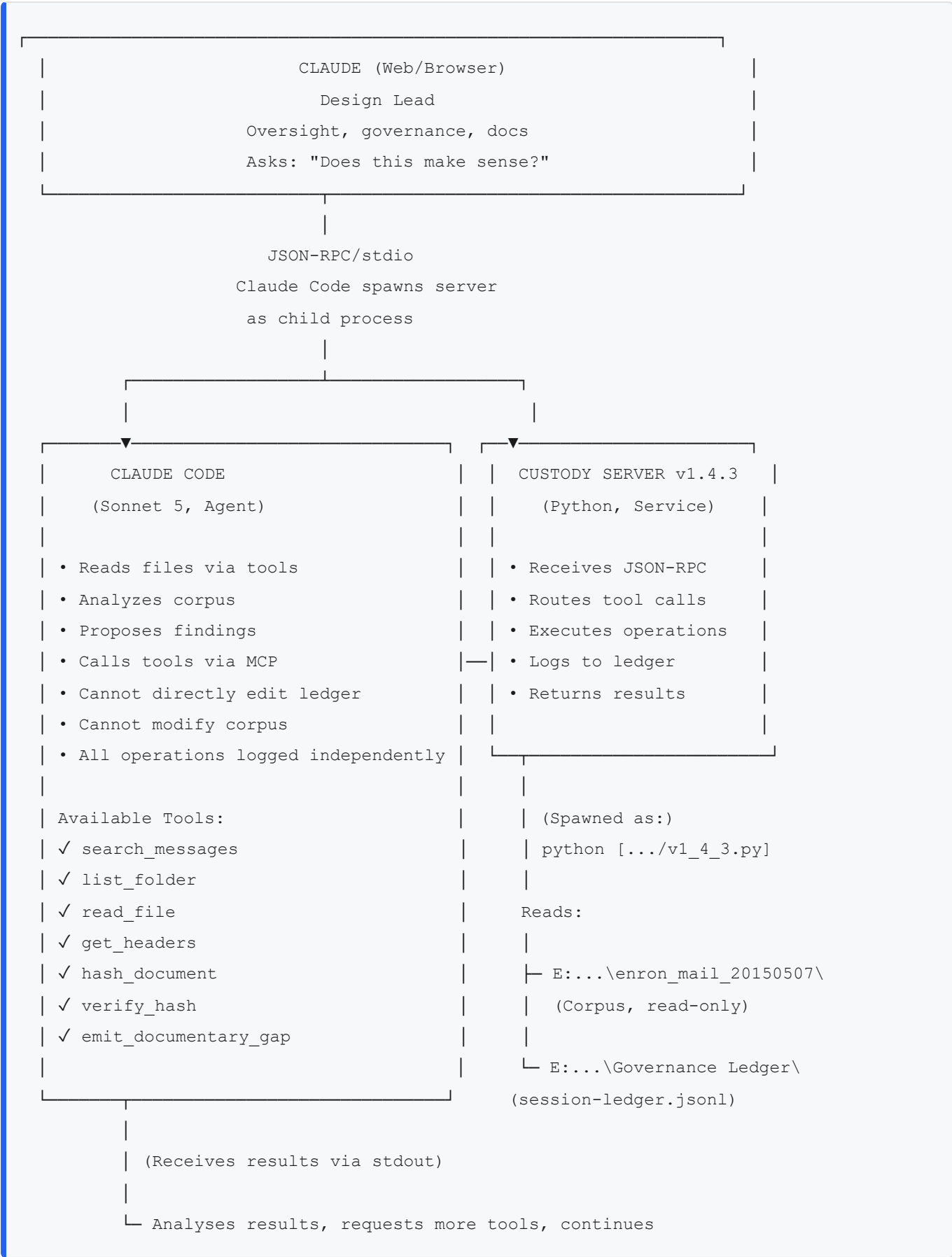
Requirement: Easy to add new tools as needs evolve.

Implementation:

- Add method to CustodyServer class
- Register in TOOLS_REGISTRY (JSON schema)
- Add handler in JSONRPCServer.handle_tool_call()
- Done

Example: If we later need `redact_file()` or `cluster_messages()`, just add the method and register it.

COMPONENT INTERACTION DIAGRAM



OPERATIONAL CHECKLIST

Daily Startup

- ☐ Claude Code runs normally
- ☐ `claude mcp list` shows dossier-custody as Connected
- ☐ Can call tools: `list_folder kaminski-v` should return file list
- ☐ Ledger file exists and is writable: `E:...\Governance Ledger\session-ledger.jsonl`

After Any Change

- ☐ Syntax check: `python dossier_custody_server_v1_4_3.py` (should start, then Ctrl+C)
- ☐ Re-register: `claude mcp add dossier-custody python "E:..."`
- ☐ Restart Claude Code: `/exit` then `claude`
- ☐ Verify tools: `list dossier-custody tools` (should show 7)
- ☐ Test call: `search_messages kaminski-v test` (should log action to ledger)

Before Investigation Session

- ☐ Ledger is readable: `Get-Content session-ledger.jsonl -Tail 1` (shows recent entry)
 - ☐ Session ID documented
 - ☐ Starting action counter noted (for reference)
 - ☐ Corpus integrity verified (optional: spot-check one file hash)
-

SUMMARY: Why We Have Each Component

Component	Purpose	Why Critical
ENRON Corpus	Test data for evidence analysis	Enables pilot without risking real data
Governance Ledger	Immutable audit trail	Proves chain of custody for SCL paper
Custody Server v1.4.3	Mediates corpus access + logs	Prevents agent from hiding operations
LedgerWriter	Append-only persistence	Fail-closed semantics, no silent failures
MCP Registration (claude.json + <code>mcp add</code>)	Tells Claude Code how to launch server	Required for tool visibility
Version History	Audit trail of changes	Enables rollback, documents evolution
CIPDA Framework	Session structure	Makes diagnosis systematic & repeatable

This is the complete infrastructure. Every file, every log, every connection explained and rationalized.

Next Session (Tomorrow): Path productisation (make portable) + Leader panel (multi-model testing).